Quantum Hypergraph Partitioning

Yiran Li
University of Toronto
Toronto, Ontario, Canada
one.li@utoronto.ca

Y. Batuhan Yilmaz
yildirim.yilmaz@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Michael Silver
University of Toronto
Toronto, Ontario, Canada
m.silver@mail.utoronto.ca

Zachary Vernec
zachary.vernec@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Hans-Arno Jacobsen
jacobsen@eecg.toronto.edu
University of Toronto
Toronto, Ontario, Canada

arXiv:submit/7539786 [cs.SI] 1 May 2026

Abstract

Hypergraph partitioning is a fundamental optimization problem with applications in data management and other domains involving higher-order relations. In this paper, we study balanced hypergraph partitioning from the perspective of quantum optimization. We formalize balanced k-way hypergraph partitioning with general hyperedge cut functions, and derive corresponding binary optimization formulations targeted at quantum optimization methods in both the two-way and multi-way settings. Our discussion highlights which cut functions admit Quadratic Unconstrained Binary Optimization (QUBO) encodings and which instead lead to higher-order binary objectives or rational forms. As a preliminary empirical validation, we focus on balanced two-way partitioning with the all-or-nothing cut on 3-uniform hypergraphs, where a direct QUBO is available, and evaluate simulated Quantum Approximate Optimization Algorithm (QAOA) and Simulated Annealing (SA) on small instances against exact solutions. The results show that the formulation is effective on small hypergraphs and that the balance-penalty weight plays a critical role in trading off cut quality and balance.

CCS Concepts

• **Mathematics of computing** → **Hypergraphs**; • **Computer systems organization** → **Quantum computing**.

Keywords

Quantum Computing, Hypergraph, Partitioning, QAOA, QUBO

ACM Reference Format:
Yiran Li, Y. Batuhan Yilmaz, Michael Silver, Zachary Vernec, and Hans-Arno Jacobsen. 2026. Quantum Hypergraph Partitioning. In *The third workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '26), May 31-June 05, 2026, Bengaluru, India.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3811628.3811836

1 Introduction

Hypergraphs generalize traditional graphs by allowing a hyperedge to connect more than two nodes, making them a natural model for multi-way relationships and higher-order interactions such as group chats in social networks, multi-protein interactions in biological systems, and multi-qubit gates in quantum circuits. Hypergraph partitioning aims to divide the nodes in a hypergraph into a designated number of subsets, such that the hyperedge connections among them are minimized. Aside from its applications to simplifying sparse-matrix multiplications [12] and analyzing biological networks [21], hypergraph partitioning has been found useful for distributing quantum circuits [4], as well as data management problems such as optimizing matrix storage layout [62] and storage sharding for distributed databases [30].

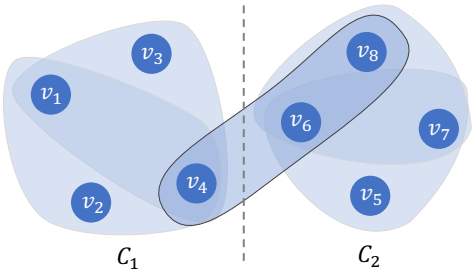


Figure 1: An example of balanced hypergraph partitioning.

Figure 1 illustrates a balanced two-way hypergraph partition, where the node set is divided into two equally sized parts C_1 and C_2, and hyperedges may either lie entirely within one partition or be cut across the two. More generally, in this work we study balanced k-way hypergraph partitioning, which seeks a partition of the node set into k disjoint subsets of nearly equal size while minimizing a chosen hypergraph cut objective. This problem is computationally challenging because, in addition to minimizing the cut objective over all node assignments, it must also enforce balance among the partition sizes. In contrast to the unbalanced case, where polynomial-time solutions are available for certain two-way cut objectives, balanced hypergraph partitioning is NP-hard even for $k = 2$ [60]. Given the difficulty of finding optimal balanced partitions, existing scalable methods typically rely on spectral relaxation [36], greedy Louvain-style methods [22], local search [30], or ensembles of heuristics [52].

Despite the success of these classical heuristics, finding high-quality partitions for increasingly large and dense hypergraphs remains a significant bottleneck due to the high-dimensional nature of the search space. Quantum computing has emerged as a promising alternative for addressing such computationally intensive optimization tasks [2]. Fundamental building blocks of quantum computers are qubits, which can exist in superpositions of basis

This work is licensed under a Creative Commons Attribution 4.0 International License.
Q-Data '26, Bengaluru, India
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2703-0/2026/05
https://doi.org/10.1145/3811628.3811836

states and become entangled with one another. By leveraging these uniquely quantum phenomena, certain problems can be solved with significantly lower complexity [25, 29, 53].

Recent advances in hardware have brought quantum computers closer to utility-scale [1, 9, 33]. Nevertheless, on the current Noisy Intermediate-Scale Quantum (NISQ) [47] devices, applications remain constrained by hardware limitations, including limited qubit counts, shallow circuit depths, and high error rates. While many quantum algorithms require fault-tolerant hardware, quantum annealing (QA) [19, 26, 42, 43, 48] and variational quantum algorithms (VQAs) [5, 8, 11, 13, 28, 41, 44, 54] such as the Quantum Approximate Optimization Algorithm (QAOA) [20] offer a viable path for the NISQ era. These frameworks map combinatorial optimization problems onto quantum operators, which steer the evolution of the quantum state such that the optimal solution is encoded in the final measurement with high probability. Both QA and QAOA have been applied to various graph-optimization problems such as travelling salesman problem, maximum cut, and minimum k-way partitioning [3, 15, 16, 35, 38, 40, 45, 51, 55].

In this paper, we study balanced hypergraph partitioning from the perspective of quantum optimization, with formulations targeted at both QAOA and quantum annealing. We derive and discuss binary optimization formulations for a range of hypergraph cut objectives from the literature, covering both two-way and multi-way settings. As a preliminary empirical validation, we focus on the all-or-nothing cut on 3-uniform hypergraphs, where a quadratic formulation is available, and evaluate the resulting formulation using simulated QAOA and simulated annealing [7, 34, 59] against exact solutions. Our results provide initial evidence that the proposed formulation is effective on small instances and highlight the importance of the balance-penalty coefficient in trading off feasibility and cut quality.

2 Preliminaries

Quadratic Unconstrained Binary Optimization (QUBO). QUBO is a unifying framework for combinatorial optimization and a standard input format for Ising-model-based solvers. It can represent diverse NP-hard problems, such as partitioning, routing, and scheduling, in a common mathematical form without explicit constraints. Instead, constraints are absorbed into the objective as penalty terms, turning the problem into energy minimization, where the global minimum corresponds to the optimum of the original formulation.

Formally, a QUBO problem is defined as the minimization of a quadratic objective function:

$$\min_{x\in\{0,1\}^n} \mathbf{x}^\mathsf{T}\mathbf{Q}\mathbf{x}, \tag{1}$$

where $\mathbf{x}=(x_1,\ldots,x_n)$ is a vector of n binary variables and $\mathbf{Q}\in\mathbb{R}^{n\times n}$ is a real symmetric matrix. The diagonal entries Q_{ii} represent the coefficients of linear terms, since $x_i^2=x_i$, while the off-diagonal entries Q_{ij} encode quadratic interactions between variables. Constructing an appropriate matrix $\mathbf{Q}$ is the central task in QUBO mapping, as it must balance the original objective against penalty terms that discourage the violation of problem constraints.

A natural generalization of QUBO is Higher-order Unconstrained Binary Optimization (HUBO), in which the objective may contain terms of degree greater than two. HUBO objectives can often be handled directly in gate-based settings such as QAOA, whereas QA typically requires a prior reduction to quadratic form.

Quantum optimization. QA and QAOA represent two of the most widely used paradigms for quantum optimization on NISQ-era hardware. Both approaches encode a combinatorial optimization problem into a problem Hamiltonian H_P, whose ground state corresponds to the optimal solution. Specifically, QUBO problems can be mapped to an Ising Hamiltonian

$$H_I=-\sum_i h_i\sigma_i^z-\sum_{i,j}J_{ij}\sigma_i^z\sigma_j^z, \tag{2}$$

where σ_i^z is a Pauli-z operator for the ith spin, h_i represents the local longitudinal fields, and J_{ij} denotes the pairwise coupling strengths. The mapping is done by transforming binary variables $x_i\in\{0,1\}$ into spins $z_i\in\{-1,+1\}$ via the mapping $z_i=2x_i-1$, and setting $h_i=\mathbf{Q}_{ii}/2$ and $J_{ij}=\mathbf{Q}_{ij}/4$. The optimization task is thus equivalent to finding the ground state, the state that minimizes the expectation value

$$\min_{|\mathbf{x}\rangle}\langle\mathbf{x}|H_I|\mathbf{x}\rangle=\min_{\mathbf{z}\in\{-1,+1\}^n}\left(-\sum_i h_i z_i-\sum_{i,j}J_{ij}z_iz_j\right). \tag{3}$$

QA implements this optimization through a continuous adiabatic evolution of an Ising spin system. The system begins in the ground state $|\psi_D\rangle$ of a driver Hamiltonian H_D that does not commute with H_P and is slowly evolved toward the problem Hamiltonian, ideally remaining in its ground state throughout the transition. A typical choice for the driver Hamiltonian is $H_D=-\sum_i\sigma_i^x$, defined as the negative sum of Pauli-X operators acting on each qubit. Its ground state is the uniform superposition of all 2^n computational basis states,

$$|\psi_D\rangle=\frac{1}{\sqrt{2^n}}\sum_{\mathbf{x}}|\mathbf{x}\rangle. \tag{4}$$

The total Hamiltonian of QA is given by $H(t)=A(t)H_D+B(t)H_P$, where $A(t)$ and $B(t)$ are function of time t that control the transition from H_P and to H_D. At the initial time $t=t_i$, $H(t_i)=H_D$, and at the final time, $H(t_f)=H_P$. As long as the Hamiltonian transition is sufficiently slow, the system's state $|\psi(t)\rangle$ closely follows the instantaneous ground state. Thus, the final state $|\psi(t_f)\rangle$ provides a close approximation of the ground state $|\psi_P\rangle$ of H_P.

QAOA provides a digital, gate-based approach. Under certain conditions, QAOA is a discrete approximation of QA [10]. However, QAOA can simulate the evolution of a broader class of Hamiltonians. The evolution is discretized into L alternating applications of the problem Hamiltonian H_P and a mixing Hamiltonian H_M. The state is initialzed as the ground state $|\psi_M\rangle$ of H_M, and its evolution is governed by variational parameters $\boldsymbol{\beta}=(\beta_1,\ldots,\beta_L)\in\mathbb{R}^L$ and $\boldsymbol{\gamma}=(\gamma_1,\ldots,\gamma_L)\in\mathbb{R}^L$:

$$|\boldsymbol{\beta},\boldsymbol{\gamma}\rangle=\prod_{l=1}^{L}e^{-i\beta_l H_M}e^{-i\gamma_l H_P}|\psi_M\rangle. \tag{5}$$

At each step of the classical optimization loop, the state $|\boldsymbol{\beta},\boldsymbol{\gamma}\rangle$ is measured in the computational basis to extract a bit-string $\mathbf{z}$. This result is used to evaluate the objective function and guide the classical optimizer in updating $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$. With increasing number of K and optimization iterations, $|\boldsymbol{\beta},\boldsymbol{\gamma}\rangle$ converges to the ground state $|\psi_P\rangle$ of H_P.

Both QA and QAOA are probabilistic protocols, as the final state is generally a superposition of multiple computational basis states. Measuring this state yields a single bit-string z with a probability determined by the square of its corresponding state's amplitude.

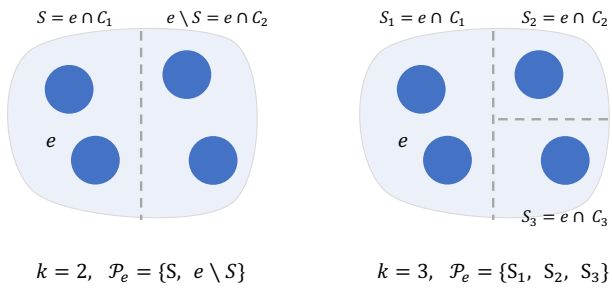


Figure 2: Illustration of the partition induced on hyperedge e.

Consequently, the success of the protocol is assessed by the success probability, the likelihood of sampling a bit-string that represents an optimal or near-optimal solution. In practice, the algorithm is typically executed multiple times to ensure a high confidence of obtaining the correct outcome.

Hypergraph. A hypergraph is defined as $H = (V,E)$, where $V = \{v_1,\ldots,v_n\}$ is the set of n nodes and $E = \{e_1,\ldots,e_m\}$ is a collection of m hyperedges, with each hyperedge $e \in E$ being an arbitrary subset of V, i.e., $E \subseteq 2^V$. Each hyperedge e is associated with a positive weight $w_e > 0$; in the unweighted case, $w_e = 1$. The degree of a node v, denoted by $d(v)$, is the number of hyperedges incident to v. A commonly studied special case is the r-uniform hypergraph, in which every hyperedge has size r.

3 Problem Definition

We study the balanced k-way hypergraph partitioning problem. The goal is to assign each *node* $v \in V$ to one of k disjoint nonempty partitions $C = \{C_1,\ldots,C_k\}$ such that $\bigcup_{i=1}^{k} C_i = V$ and $C_i \cap C_j = \varnothing$ for $i \neq j$, while keeping the partition sizes as even as possible.

Each hyperedge e inherits a partition from the node partition. In general, its incident nodes are divided into disjoint nonempty subsets $S_1,\ldots,S_r$, where $1 \le r \le k$, such that $e = \bigcup_{i=1}^{r} S_i$. We denote the partition of e induced by the node partition C as $\mathcal{P}_e(C) = \{S_1,\ldots,S_r\}$, or simply $\mathcal{P}_e$. For two-way partitioning, this reduces to $\mathcal{P}_e = \{S, e \setminus S\}$ for some $S \subseteq e$. Figure 2 illustrates these notations: in the two-way case, the hyperedge is split into two parts, $S = e \cap C_1$ and $e \setminus S = e \cap C_2$; in the multi-way case, the induced partition consists of the nonempty intersections with the global partitions, e.g., $\mathcal{P}_e = \{S_1, S_2, S_3\}$ for $k = 3$.

The contribution of hyperedge e to the partitioning objective is measured by a hyperedge cut function $f(\mathcal{P}_e)$, or $f(S)$ in the two-way case. Various choices of this cut function have been studied in the literature; we refer readers to [60] for a review. A formal problem definition is as follows.

Definition 3.1 (Balanced k-way hypergraph partitioning). Given a weighted hypergraph $H = (V,E)$ with hyperedge weights w_e and a hyperedge cut function f, the balanced k-way hypergraph partitioning problem is to find a partition $C = \{C_1,\ldots,C_k\}$ of V that minimizes

$$\begin{aligned} \min_{C=\{C_1,\ldots,C_k\}} \quad & \sum_{e \in E} w_e f(\mathcal{P}_e(C)) \\ \text{s.t.} \quad & \bigcup_{i=1}^{k} C_i = V, \quad C_i \cap C_j = \varnothing \quad (i \neq j), \\ & C_i \neq \varnothing \quad \text{for all } i, \\ & \big||C_i| - |C_j|\big| \le 1 \quad \text{for all } i,j \in \{1,\ldots,k\}, \end{aligned} \tag{6}$$

where $\mathcal{P}_e(C)$ denotes the partition of hyperedge e induced by C, i.e.,

$$\mathcal{P}_e(C) = \{e \cap C_i \mid e \cap C_i \neq \varnothing, i = 1,\ldots,k\}.$$

In this optimization problem, the first three constraints ensure a valid partition of the node set, i.e., each node belongs to exactly one partition and no partition is empty (*partition constraint*). The last constraint enforces balance by requiring all partitions to have the same size when k divides $n = |V|$, and otherwise to differ in size by at most one (*balance constraint*).

Theoretical complexity. The complexity of hypergraph partitioning depends strongly on both the number of partitions and the choice of the cut function. For two-way partitioning ($k = 2$) without a balance constraint, cardinality-based submodular cut functions, including several standard choices considered in this work, admit polynomial-time solutions via reduction to graph cut problems [60]. For multiway partitioning with $k > 2$ and no balance constraint, the problem is NP-hard when k is part of the input, although an exact algorithm with running time $n^{O(k)}$ has been given [14]. Once balance constraints are imposed, however, hypergraph partitioning becomes NP-hard for any $k \ge 2$ by reduction from the minimum graph bisection problem [24]. Therefore, the problem studied in this work, namely balanced hypergraph partitioning, is NP-hard.

4 Objective Formulations

For the hypergraph partitioning problem in Eq. 6, the corresponding binary optimization objective must encode both the cut cost and the partitioning constraints. Depending on the chosen cut function, this objective may take either a quadratic form, yielding a QUBO, or a higher-order form, yielding a HUBO. We therefore decompose the objective into three components:

$$E(\mathbf{x}) = E_{\text{cut}}(\mathbf{x}) + \alpha E_{\text{partition}}(\mathbf{x}) + \lambda E_{\text{balance}}(\mathbf{x}), \tag{7}$$

where E_{cut} represents the chosen hyperedge cut objective, $E_{\text{partition}}$ enforces that the binary assignment defines a valid partition, and E_{balance} enforces balanced partition sizes. The coefficient $\alpha > 0$ controls the strength of the partition-validity penalty, while $\lambda > 0$ is the balance-penalty coefficient.

In the following subsections, we treat two-way and multi-way partitioning separately, since they require different objective constructions for encoding partition assignments and constraints, and some cut objectives are specific to one setting or take substantially different forms across the two. Accordingly, we derive the corresponding forms of E_{cut}, $E_{\text{partition}}$, and E_{balance} for each case.

4.1 Two-way Partitioning ($k = 2$)

For $k = 2$, each vertex $v_i \in V$ is associated with a single binary variable $x_i \in \{0,1\}$ indicating which partition it belongs to. We interpret $x_i = 1$

as assigning v_i to the first partition and $x_i = 0$ to the second. The resulting QUBO therefore uses n binary variables, i.e., $\mathbf{x} \in \{0,1\}^n$.

Under this encoding, the validity constraints are satisfied implicitly: each vertex is assigned to exactly one of the two partitions, so the two blocks are automatically disjoint and their union is exactly V. Hence, no additional validity penalty is required, and we set $E_{\text{partition}}(\mathbf{x}) = 0$ and $E(\mathbf{x}) = E_{\text{cut}}(\mathbf{x}) + \lambda E_{\text{balance}}(\mathbf{x})$.

For balanced partitioning, we use the constraint term

$$E_{\text{balance}}(\mathbf{x}) = \left(\sum_{i=1}^{n} x_i - \frac{n}{2}\right)^2, \tag{8}$$

which encourages the two partition sizes to be as equal as possible. This term is minimized when the partitions have equal size if n is even, and differ by at most one if n is odd. In particular, it guarantees that both partitions are nonempty for $n \geq 2$.

For hypergraph partitioning without an explicit balance requirement, nonemptiness is a weaker condition than the above balance constraint. Nevertheless, the same penalty in Eq. (8) can still be retained with a smaller coefficient, so that it serves as a soft regularizer against trivial collapsed assignments while allowing greater flexibility in partition sizes. We therefore adopt Eq. (8) as the constraint term for two-way partitioning, with the penalty weight λ chosen according to whether strict balance enforcement or only mild regularization is desired.

We next discuss the cut term $E_{\text{cut}}(\mathbf{x})$ for several hyperedge cut models, beginning with the all-or-nothing cut. In the formulation, S denotes the larger part of the hyperedge e after the cut.

All-or-nothing cut ($k = 2$). This cut function penalizes all hyperedge cuts by a uniform cost:

$$f_{\text{AoN}}(S) = \begin{cases} 0, & \text{if } S = e, \\ 1, & \text{otherwise.} \end{cases} \tag{9}$$

An unconstrained binary optimization objective for the all-or-nothing cut is given in [49] as

$$E_{\text{AoN}}(\mathbf{x}) = \sum_{e \in E} \left(1 - \prod_{v_i \in e} x_i - \prod_{v_i \in e} (1 - x_i)\right). \tag{10}$$

For each hyperedge e, the term $\prod_{v_i \in e} x_i$ equals 1 only when all vertices $v_i \in e$ are assigned to partition 1, while $\prod_{v_i \in e}(1 - x_i)$ equals 1 only when all vertices $v_i \in e$ are assigned to partition 0. Therefore, the expression contributes 0 when all vertices in e lie in the same partition and 1 otherwise, which matches the all-or-nothing cut objective.

For 3-uniform hypergraphs, Eq. (10) reduces to a valid QUBO, since the cubic terms from the two products cancel and the resulting expression reduces to a quadratic polynomial. However, for hypergraphs with larger hyperedges, Eq. (10) becomes a HUBO rather than a QUBO. This is not a major issue for gate-based approaches such as QAOA, which can in principle optimize higher-order objectives directly, although larger hyperedges generally lead to deeper circuits. In contrast, quantum annealing requires a quadratic form, so for $|e| > 3$ the objective must first be quadratized [39], typically by introducing auxiliary variables.

Quadratic penalty. This cut function assigns a cost equal to the product of the sizes of the two parts induced on a hyperedge:

$$f_{\text{quadratic}}(S) = |S| \cdot |e \setminus S|. \tag{11}$$

For a hyperedge e, let $S = \{v_i \in e \mid x_i = 1\}$. Then $\sum_{v_i, v_j \in e} x_i(1 - x_j)$ counts exactly the ordered pairs (v_i, v_j) such that $v_i \in S$ and $v_j \in e \setminus S$, and therefore equals $|S| \cdot |e \setminus S|$. Summing over all hyperedges gives the QUBO objective

$$E_{\text{quadratic}}(\mathbf{x}) = \sum_{e \in E} \sum_{v_i, v_j \in e} x_i(1 - x_j). \tag{12}$$

Linear penalty. This cut function penalizes a cut on hyperedge e by the size of the smaller side:

$$f_{\text{linear}}(S) = \min(|S|, |e \setminus S|). \tag{13}$$

Let $s = |S|$ and $t = |e|$. Since $|e \setminus S| = t - s$, we can rewrite the penalty as

$$f_{\text{linear}}(S) = \min(s, t - s) = \frac{t - |2s - t|}{2} = -\left|s - \frac{t}{2}\right| + \frac{t}{2}. \tag{14}$$

Because of the absolute value term, this function is not directly representable as a QUBO.

Hypergraph normalized cut [63]. In the two-way setting, let $\{C_1, C_2\}$ denote the partition of the vertex set V, and let $S = e \cap C_1$, so that $e \setminus S = e \cap C_2$. Then the contribution of hyperedge e can be written as

$$f_{\text{Ncut}}(S) = \left(\frac{1}{\text{vol}(C_1)} + \frac{1}{\text{vol}(C_2)}\right) \frac{|S||e \setminus S|}{|e|}, \tag{15}$$

where $\text{vol}(C) = \sum_{v_i \in C} d(v_i)$ denotes the volume of a vertex set C.

The term $|S||e \setminus S|/|e|$ is a scaled version of the quadratic penalty. Therefore, if the normalization factors $1/\text{vol}(C_1)$ and $1/\text{vol}(C_2)$ are omitted, the remaining objective is equivalent in form to the quadratic penalty introduced above, up to the hyperedge-dependent scaling factor $1/|e|$, which can be absorbed into the hyperedge weights.

However, the full normalized cut objective is not directly representable as a QUBO or HUBO. Since $\text{vol}(C_1)$ and $\text{vol}(C_2)$ both depend on the binary assignment variables, the resulting objective is a rational function.

4.2 Multi-way Partitioning ($k > 2$)

For $k > 2$, each node v_i is represented by a one-hot vector of k binary variables $(x_{i1}, \ldots, x_{ik})$, where $x_{ic} = 1$ indicates assignment to partition c. The QUBO thus uses nk binary variables. The balance penalty is

$$E_{\text{balance}}(\mathbf{x}) = \sum_{c=1}^{k} \left(\sum_{i=1}^{n} x_{ic} - \frac{n}{k}\right)^2, \tag{16}$$

which enforces that all partitions have nearly equal size.

An additional constraint ensures that each node belongs to one partition only:

$$E_{\text{partition}}(\mathbf{x}) = \sum_{i=1}^{n} \left(\sum_{c=1}^{k} x_{ic} - 1\right)^2 \tag{17}$$

Let $\mathcal{P}(e) = \{S_1, \ldots, S_r\}$ denote the nonempty parts cut from hyperedge e. We next discuss different multi-way cut objectives and their corresponding binary optimization formulations.

All-or-nothing cut. Extending Eq. (9) and Eq. (10) to the multi-way setting, the all-or-nothing cut assigns a unit cost whenever a hyperedge spans more than one partition, and zero otherwise. Formally,

$$f_{\text{AoN}}(\mathcal{P}_e) = \begin{cases} 0, & \text{if } |\mathcal{P}_e| = 1, \\ 1, & \text{otherwise.} \end{cases} \tag{18}$$

That is, a hyperedge incurs no penalty if all of its vertices lie in the same partition, and incurs a penalty of 1 as soon as it is split.

Under the one-hot encoding, the corresponding HUBO objective is

$$E_{\text{AoN}}(\mathbf{x}) = \sum_{e\in E}\left(1-\sum_{c=1}^{k}\prod_{v_i\in e} x_{ic}\right), \tag{19}$$

where $\prod_{v_i\in e} x_{ic}$ equals 1 if and only if all vertices in hyperedge e are assigned to partition c, and equals 0 otherwise. Hence, the inner summation is 1 exactly when all vertices of e belong to a single partition, and 0 otherwise, which matches the definition in Eq. (18).

***K*−1 penalty.** This cut function assigns a cost equal to the number of partitions spanned by a hyperedge minus one:

$$f_{K-1}(\mathcal{P}_e) = |\mathcal{P}_e| - 1. \tag{20}$$

Thus, a hyperedge incurs zero cost when all of its vertices lie in the same partition, and the penalty increases linearly with the number of partitions it touches. In the special case of two-way partitioning ($k=2$), this reduces to the all-or-nothing cut.

Under the one-hot encoding, the corresponding HUBO objective can be written as

$$E_{K-1}(\mathbf{x}) = \sum_{e\in E}\left(\sum_{c=1}^{k}\left(1-\prod_{v_i\in e}(1-x_{ic})\right)-1\right). \tag{21}$$

For each hyperedge e and partition c, the term $1-\prod_{v_i\in e}(1-x_{ic})$ indicates whether e has at least one vertex assigned to partition c. Summing over all c therefore counts the number of partitions spanned by e, and subtracting 1 yields the K-1 penalty.

Hypergraph normalized cut. The normalized cut objective can also be extended to the multi-way setting [61, 63]:

$$f_{\text{Ncut}}(\mathcal{P}_e) = \sum_{i=1}^{k}\frac{|e\cap C_i||e\setminus C_i|}{\text{vol}(C_i)|e|}, \tag{22}$$

where $\text{vol}(C_i) = \sum_{v_j\in C_i} d(v_j)$ denotes the volume of partition C_i.

As in the two-way case, the numerator is a scaled quadratic-type term, but the denominator depends on the partition volumes. Substituting binary assignment variables therefore yields a rational function rather than a polynomial. Hence, this cut function does not admit a direct QUBO or HUBO formulation.

Nevertheless, since this work focuses on balanced hypergraph partitioning, the partition volumes $\text{vol}(C_i)$ are expected to be similar across clusters. Under this assumption, the normalization factors $1/\text{vol}(C_i)$ may be viewed as approximately constant, leading to the following unnormalized surrogate:

$$f_{\text{quadratic}}(\mathcal{P}_e) = \sum_{i=1}^{k}\frac{|e\cap C_i||e\setminus C_i|}{|e|}, \tag{23}$$

which is the natural multi-way extension of the quadratic penalty. Under one-hot encoding, this yields the QUBO term

$$E_{\text{quadratic}}(\mathbf{x}) = \sum_{e\in E}\sum_{c=1}^{k}\frac{1}{|e|}\sum_{v_i,v_j\in e} x_{ic}(1-x_{jc}). \tag{24}$$

Indeed, for each hyperedge e and partition C, the inner summation counts the ordered pairs (v_i,v_j) such that $v_i\in e\cap C$ and $v_j\in e\setminus C$, and is therefore equal to $|e\cap C||e\setminus C|$. Thus, Eq. (24) is exactly the QUBO corresponding to the unnormalized version of Eq. (22). When $k=2$, this reduces to the two-way quadratic QUBO in Eq. (12), up to the hyperedge-dependent scaling factor $1/|e|$.

Hypergraph random walk conductance. This recent hypergraph clustering objective is based on given transition probabilities for a random walk over the hypergraph, with $p(v_i,v_j)$ denoting the given probability that a random walk starting from v_i ends at v_j [36]. For a partition $\mathcal{P}=\{C_1,\ldots,C_k\}$, the objective to be minimized is

$$\Phi_{\text{HRWC}}(\mathcal{P}) = \frac{1}{k}\sum_{C\in\mathcal{P}}\frac{1}{|C|}\sum_{v_i\in C}\sum_{v_j\notin C} p(v_i,v_j). \tag{25}$$

Under balanced k-way partitioning, we have $|C|=n/k$ for every cluster C. Hence, the normalization factor $1/|C|$ becomes a constant, and Eq. (25) reduces to

$$\Phi_{\text{HRWC}}(\mathcal{P}) = \frac{1}{n}\sum_{C\in\mathcal{P}}\sum_{v_i\in C}\sum_{v_j\notin C} p(v_i,v_j). \tag{26}$$

Using one-hot variables $x_{ic}\in\{0,1\}$, where $x_{ic}=1$ indicates that vertex v_i is assigned to partition C_c, the corresponding QUBO term can be written as

$$E_{\text{HRWC}}(\mathbf{x}) = \frac{1}{n}\sum_{v_i,v_j\in V} p(v_i,v_j)\sum_{c=1}^{k} x_{ic}(1-x_{jc}). \tag{27}$$

For any pair (v_i,v_j), the inner summation $\sum_{c=1}^{k} x_{ic}(1-x_{jc})$ equals 1 if v_i and v_j are assigned to different partitions, and 0 otherwise. Thus, Eq. (27) measures the total random-walk escape probability across clusters. This formulation is also similar in form to the multi-way quadratic QUBO in Eq. (24); the key difference is that here the coefficients are specified by the random-walk transition probabilities $p(v_i,v_j)$, rather than by hyperedge membership. Since $p(v_i,v_j)$ is treated as a given constant, Eq. (27) is a valid quadratic binary formulation.

Alternative binary encoding. Instead of one-hot encoding, a k-way partition can be represented using only $\lceil\log_2 k\rceil$ binary variables per vertex, by encoding each partition index in binary. This reduces the number of variables from nk to $n\lceil\log_2 k\rceil$.

However, this saving comes at the cost of higher-order terms. Even identifying whether a vertex belongs to a specific partition requires a term of degree $\lceil\log_2 k\rceil$. Consequently, both the balance term and the cut objectives involve higher-order products, yielding HUBO formulations rather than QUBOs. Such terms cannot be handled directly by quantum annealers without quadratization, and for QAOA they generally imply deeper circuits and more costly implementations. Therefore, although binary encoding uses fewer qubits, we consider it less practical than one-hot encoding for this work.

5 Related Work

Hypergraph Partitioning. Classical practical approaches include multilevel partitioners for circuit and graph applications [32, 52], local-search-based methods for large social networks [30], and modularity clustering methods [22, 23]. For hypergraph clustering, prior work has also generalized normalized cut and related spectral objectives to hypergraphs and attributed hypergraphs, leading to spectral or matrix-factorization-based algorithms [36, 61, 63]. On the theoretical side, Veldt et al. [60] provide a general treatment of hyperedge cut functions and the complexity of the corresponding partitioning problems, while more recent work studies related diffusion- and conductance-based objectives for local clustering [31].

Quantum (Hyper)graph Partitioning. Prior quantum work has focused mainly on graph partitioning, including quantum annealing formulations for graph bisection and their multilevel extensions [57, 58], QUBO formulations for constrained cut variants [3, 17], and QAOA-based approaches to application-specific settings such as image segmentation [56]. Other related directions include multi-way graph partitioning with qudits [46] and learning-based selection of balance-penalty coefficients for minimum bisection [50]. In contrast, quantum algorithms for hypergraph partitioning remain largely unexplored. To our knowledge, the only directly related prior work is that of Rodriguez [49], which provides a HUBO formulation for hypergraph two-way partitioning under the all-or-nothing objective. In contrast, we derive QUBO/HUBO formulations for both two-way and multi-way partitioning under general hyperedge cut functions. We also note related quantum approaches on hypergraphs that are orthogonal to our focus, including quantum sparsification methods [37] and theoretical analyses of QAOA on hypergraph-structured optimization problems [6], which do not provide direct formulations for hypergraph partitioning.

6 Experiments

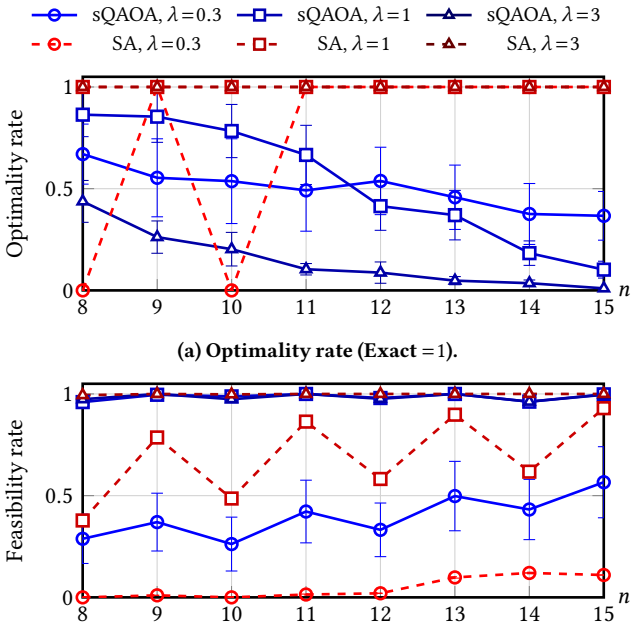
We present a preliminary experiment of balanced two-way partitioning with the all-or-nothing cut on hypergraphs. Our evaluation focuses on this objective function for 3-uniform hypergraphs, since this setting admits a direct QUBO formulation that can be easily simulated. Restricting to small instances also enables exact verification via exhaustive search. Code and data are available at https://github.com/CyanideCentral/quantum-hypergraph-partitioning.

Settings. We generate 100 random hypergraphs for each $n \in \{8,\ldots,15\}$ as our dataset. Each hypergraph is unweighted, connected, and has uniform hyperedge size 3 and average node degree $\bar{d} \approx 5$. We compare four methods: exhaustive search (Exact), KaHyPar [52], simulated QAOA (sQAOA) with X-mixer Hamiltonian $H_M = -\sum_i \sigma_i^x$, and simulated annealing (SA). For sQAOA, we use the statevector simulator in Qiskit [27] with circuit depth 1, and select the best balanced partition among the top-10 states with highest probability. For SA, we use the simulated annealing sampler in Ocean SDK [18] with 100 reads. For both sQAOA and SA, we evaluate balance-penalty coefficients $\lambda \in \{0.3, 1, 3\}$. Each method is run 5 times with distinct random seeds; we report averages and standard errors across these runs.

Evaluation Protocol. We evaluate each solver on the all-or-nothing cut objective (E_{AoN} in Eq. (10)), which is a valid QUBO for size-3 hyperedges, over all hypergraph instances in the dataset. Since our target problem is balanced partitioning, we report both *feasibility rate*, defined as the fraction of returned partitions that satisfy the balance constraint, and *optimality rate*, defined as the fraction of feasible partitions whose all-or-nothing cut value matches the optimum computed by Exact.

Partitioning Performance. Figure 3 shows the feasibility and optimality rates across different problem sizes and balance-penalties. KaHyPar attains the optimal balanced all-or-nothing cut on all tested instances, matching Exact in this small-scale setting.

sQAOA with $\lambda = 1$ performs well on smaller instances, producing optimal balanced partitions for over 90% of the cases with $n < 10$. However, its performance degrades as the hypergraph size increases.



(a) Optimality rate (Exact = 1).

(b) Feasibility rate (Exact = 1).

Figure 3: Preliminary results on 3-uniform hypergraphs with different sizes. Error bars show standard error across five runs.

With $\lambda = 0.3$, sQAOA maintains a moderate optimality rate but often violates the balance constraint. The large variation across runs suggests that sQAOA is sensitive to random initialization.

In contrast, SA with $\lambda = 3$ consistently returns optimal balanced partitions for almost all tested instances. Reducing the penalty weight to $\lambda = 1$ lowers the feasibility rate, although the returned solutions remain optimal whenever they are feasible. With $\lambda = 0.3$, SA rarely produces balanced partitions. SA also shows greater robustness across random seeds, with only negligible standard errors for both metrics.

Effect of the Balance Penalty. The coefficient λ in Eq. (7) controls the tradeoff between cut minimization and balance enforcement. When λ is too small, the solver may favor lower-cut but imbalanced solutions, as seen for SA with $\lambda = 0.3$, which usually fails to satisfy the balance constraint. On the other hand, overly large λ may overemphasize the balance term and reduce solution quality, as illustrated by the sharp drop in the optimality rate of sQAOA when λ is increased to 3.

Overall, these results suggest that moderate values of λ provide the best tradeoff for this formulation, although the preferred range may depend on both the solver and the instance distribution.

7 Conclusions

We studied balanced hypergraph partitioning as a target problem for quantum optimization. In particular, we formalized the problem under general hyperedge cut functions and derived corresponding binary optimization formulations for both two-way and multi-way partitioning. The analysis shows that some widely used objectives, such as quadratic-style cut functions, admit direct QUBO encodings, while others require higher-order formulations or approximations.

As a preliminary evaluation, we focused on balanced two-way partitioning with the all-or-nothing cut on 3-uniform hypergraphs,

where the objective can be encoded directly as a QUBO. Experiments on small instances confirmed that the formulation can recover optimal balanced partitions under suitable solver and penalty settings, and also highlighted the importance of the balance-penalty coefficient in controlling the tradeoff between feasibility and cut quality.

Future work includes extending to multi-way partitioning, evaluating larger and more diverse hypergraphs, real-hardware experiments, and systematic strategies for choosing penalty coefficients.

Acknowledgments

This research has received funding from the research project entitled "Quantum Software Consortium: Exploring Distributed Quantum Solutions for Canada" (QSC). QSC is financed by the National Sciences and Engineering Research Council of Canada (NSERC) Alliance Consortia Quantum program under grant number ALLRP587590-23.

References

[1] 2025. Quantum error correction below the surface code threshold. *Nature* 638, 8052 (2025), 920–926.

[2] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtschi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J. Egger, Bruce G. Elmegreen, Nicola Franco, Filippo Fratini, Bryce Fuller, Julien Gacon, Constantin Gonciulea, Sander Gribling, Swati Gupta, Stuart Hadfield, Raoul Heese, Gerhard Kircher, Thomas Kleinert, Thorsten Koch, Georgios Korpas, Steve Lenk, Jakub Marecek, Vanio Markov, Guglielmo Mazzola, Stefano Mensa, Naeimeh Mohseni, Giacomo Nannicini, Corey O'Meara, Elena Peña Tapia, Sebastian Pokutta, Manuel Proissl, Patrick Rebentrost, Emre Sahin, Benjamin C. B. Symons, Sabine Tornow, Víctor Valls, Stefan Woerner, Mira L. Wolf-Bauwens, Jon Yard, Sheir Yarkoni, Dirk Zechiel, Sergiy Zhuk, and Christa Zoufal. 2024. Challenges and Opportunities in Quantum Optimization. *Nature Reviews Physics* 6, 12 (Dec. 2024), 718–735. doi:10.1038/s42254-024-00770-9

[3] Ali Abbassi, Yann Dujardin, Eric Gourdin, Philippe Lacomme, and Caroline Prodhon. 2026. Quantum Approaches to the Minimum Edge Multiway Cut Problem. In *Quantum Engineering Sciences and Technologies for Industry and Services*, Frédéric Barbaresco and François Gerin (Eds.). Springer Nature Switzerland, Cham, 284–293.

[4] Pablo Andrés-Martínez and Chris Heunen. 2019. Automated Distribution of Quantum Circuits via Hypergraph Partitioning. *Physical Review A* 100, 3 (Sept. 2019), 032308.

[5] Belal Ehsan Baaquie and Leong-Chuan Kwek. 2023. Quantum-classical hybrid algorithms. In *Quantum Computers: Theory and Algorithms*. Springer, 249–256.

[6] Joao Basso, David Gamarnik, Song Mei, and Leo Zhou. 2022. Performance and Limitations of the QAOA at Constant Levels on Large Sparse Hypergraphs and Spin Glass Models. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. 335–343.

[7] Dimitris Bertsimas and John Tsitsiklis. 1993. Simulated annealing. *Statistical science* 8, 1 (1993), 10–15.

[8] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Sumner Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermanni Heimonen, Jakob S Kottmann, Tim Menke, et al. 2022. Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics* 94, 1 (2022), 015004.

[9] Dolev Bluvstein, Simon J Evered, Alexandra A Geim, Sophie H Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, et al. 2024. Logical quantum processor based on reconfigurable atom arrays. *Nature* 626, 7997 (2024), 58–65.

[10] Sami Boulebnane, James Sud, Ruslan Shaydulin, and Marco Pistoia. 2025. Quantum approximate optimization algorithm in finite size and large depth and equivalence to quantum annealing. *arXiv preprint arXiv:2503.09563* (2025).

[11] Carlos Bravo-Prieto, Ryan LaRose, Marco Cerezo, Yigit Subasi, Lukasz Cincio, and Patrick J Coles. 2023. Variational quantum linear solver. *Quantum* 7 (2023), 1188.

[12] Umit V Catalyurek and Cevdet Aykanat. 2002. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication. *IEEE Transactions on parallel and distributed systems* 10, 7 (2002), 673–693.

[13] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, et al. 2021. Variational quantum algorithms. *Nature Reviews Physics* 3, 9 (2021), 625–644.

[14] Karthekeyan Chandrasekaran and Chandra Chekuri. 2020. Hypergraph K-Cut for Fixed k in Deterministic Polynomial Time. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*. 810–821.

[15] Gavin E Crooks. 2018. Performance of the quantum approximate optimization algorithm on the maximum cut problem. *arXiv preprint arXiv:1811.08419* (2018).

[16] William Cruz-Santos, Salvador E Venegas-Andraca, and Marco Lanzagorta. 2019. A QUBO formulation of minimum multicut problem instances in trees for D-Wave quantum annealers. *Scientific reports* 9, 1 (2019), 17216.

[17] William Cruz-Santos, Salvador E. Venegas-Andraca, and Marco Lanzagorta. 2019. A QUBO Formulation of Minimum Multicut Problem Instances in Trees for D-Wave Quantum Annealers. *Scientific Reports* 9, 1 (Nov. 2019), 17216.

[18] D-Wave Quantum Inc. 2026. Ocean SDK Documentation. https://docs.dwavequantum.com/en/latest/ocean/index.html. Version 9.3.0, accessed 2026-03-30.

[19] Arnab Das and Bikas K Chakrabarti. 2008. Colloquium: Quantum annealing and analog quantum computation. *Reviews of Modern Physics* 80, 3 (2008), 1061–1081.

[20] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).

[21] Song Feng, Emily Heath, Brett Jefferson, Cliff Joslyn, Henry Kvinge, Hugh D Mitchell, Brenda Praggastis, Amie J Eisfeld, Amy C Sims, Larissa B Thackray, et al. 2021. Hypergraph models of biological networks to identify genes critical to pathogenic viral response. *BMC bioinformatics* 22, 1 (2021), 287.

[22] Zijin Feng, Miao Qiao, and Hong Cheng. 2023. Modularity-Based Hypergraph Clustering: Random Hypergraph Model, Hyperedge-cluster Relation, and Computation. *Proc. ACM Manag. Data* 1, 3 (Nov. 2023), 215:1–215:25.

[23] Zijin Feng, Miao Qiao, Chengzhi Piao, and Hong Cheng. 2025. On Graph Representation for Attributed Hypergraph Clustering. *Proc. ACM Manag. Data* 3, 1 (Feb. 2025), 59:1–59:26.

[24] M. R. Garey, D. S. Johnson, and L. Stockmeyer. 1974. Some Simplified NP-complete Problems. In *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing (STOC '74)*. Association for Computing Machinery, New York, NY, USA, 47–63.

[25] Lov K Grover. 1998. The advantages of superposition. *Science* 280, 5361 (1998), 228–228.

[26] Philipp Hauke, Helmut G Katzgraber, Wolfgang Lechner, Hidetoshi Nishimori, and William D Oliver. 2020. Perspectives of quantum annealing: Methods and implementations. *Reports on Progress in Physics* 83, 5 (2020), 054401.

[27] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. arXiv:2405.08810 [quant-ph] doi:10.48550/arXiv.2405.08810

[28] Zhang Jiang, Eleanor G Rieffel, and Zhihui Wang. 2017. Near-optimal quantum circuit for Grover's unstructured search using a transverse field. *Physical Review A* 95, 6 (2017), 062317.

[29] Richard Jozsa and Noah Linden. 2003. On the role of entanglement in quantum-computational speed-up. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 459, 2036 (2003), 2011–2032.

[30] Igor Kabiljo, Brian Karrer, Mayank Pundir, Sergey Pupyrev, and Alon Shalita. 2017. Social Hash Partitioner: A Scalable Distributed Hypergraph Partitioner. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1418–1429.

[31] Raj Kamal and Amitabha Bagchi. 2024. A Lovász-Simonovits Theorem for Hypergraphs with Application to Local Clustering. *Proc. ACM Manag. Data* 2, 4 (Sept. 2024), 190:1–190:27.

[32] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar. 1999. Multilevel Hypergraph Partitioning: Applications in VLSI Domain. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 7, 1 (March 1999), 69–79.

[33] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout Van Den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, et al. 2023. Evidence for the utility of quantum computing before fault tolerance. *Nature* 618, 7965 (2023), 500–505.

[34] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. 1983. Optimization by simulated annealing. *science* 220, 4598 (1983), 671–680.

[35] Thomas Krauss, Joey McCollum, Chapman Pendery, Sierra Litwin, and Alan J Michaels. 2020. Solving the max-flow problem on a quantum annealing computer. *IEEE Transactions on Quantum Engineering* 1 (2020), 1–10.

[36] Yiran Li, Renchi Yang, and Jieming Shi. 2023. Efficient and Effective Attributed Hypergraph Clustering via K-Nearest Neighbor Augmentation. *Proceedings of the ACM on Management of Data* 1, 2 (June 2023), 116:1–116:23.

[37] Chenghua Liu, Minbo Gao, Zhengfeng Ji, and Mingsheng Ying. 2025. Quantum Speedup for Hypergraph Sparsification. arXiv:2505.01763 [quant-ph]

[38] Andrew Lucas. 2014. Ising formulations of many NP problems. *Frontiers in physics* 2 (2014), 74887.

[39] Avradip Mandal, Arnab Roy, Sarvagya Upadhyay, and Hayato Ushijima-Mwesigwa. 2020. Compressed Quadratization of Higher Order Binary Optimization Problems. In *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF '20)*. Association for Computing Machinery, New York, NY, USA, 126–131.

[40] Roman Martoňák, Giuseppe E Santoro, and Erio Tosatti. 2004. Quantum annealing of the traveling-salesman problem. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 5 (2004), 057701.

[41] Jarrod R McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. 2016. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics* 18, 2 (2016), 023023.

[42] Catherine C McGeoch. 2022. *Adiabatic quantum computation and quantum annealing: Theory and practice*. Springer Nature.

[43] Satoshi Morita and Hidetoshi Nishimori. 2008. Mathematical foundation of quantum annealing. *J. Math. Phys.* 49, 12 (2008).

[44] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature communications* 5, 1 (2014), 4213.

[45] Sayantan Pramanik and M Girish Chandra. 2020. Quantum-assisted graph clustering and quadratic unconstrained d-ary optimisation. *arXiv preprint arXiv:2004.02608* (2020).

[46] Sayantan Pramanik and M. Girish Chandra. 2021. Quantum-Assisted Graph Clustering and Quadratic Unconstrained D-ary Optimisation. arXiv:2004.02608 [quant-ph]

[47] John Preskill. 2018. Quantum computing in the NISQ era and beyond. *Quantum* 2 (2018), 79.

[48] Atanu Rajak, Sei Suzuki, Amit Dutta, and Bikas K Chakrabarti. 2023. Quantum annealing: An overview. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 381, 2241 (2023).

[49] J. Rodriguez. 2022. Quantum Algorithms for Hypergraph Bi-Partitioning. *23ème congrès annuel de la Société Française de Recherche Opérationnelle et d'Aide à la Décision. INSA Lyon, Villeurbanne-Lyon, France (Feb 2022), https://hal.archives-ouvertes.fr/hal-03595234* (2022).

[50] Renáta Rusnáková, Martin Chovanec, and Juraj Gazda. 2025. Quantum Annealing for Minimum Bisection Problem: A Machine Learning-based Approach for Penalty Parameter Tuning. arXiv:2509.19005 [quant-ph]

[51] Özlem Salehi, Adam Glos, and Jarosław Adam Miszczak. 2022. Unconstrained binary models of the travelling salesman problem variants for quantum optimization: Ö. Salehi et al. *Quantum Information Processing* 21, 2 (2022), 67.

[52] Sebastian Schlag, Tobias Heuer, Lars Gottesbüren, Yaroslav Akhremtsev, Christian Schulz, and Peter Sanders. 2023. High-Quality Hypergraph Partitioning. *ACM J. Exp. Algorithmics* 27 (Feb. 2023), 1.9:1–1.9:39.

[53] Peter W Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* 41, 2 (1999), 303–332.

[54] Jules Tilly, Hongxiang Chen, Shuxiang Cao, Dario Picozzi, Kanav Setia, Ying Li, Edward Grant, Leonard Wossnig, Ivan Rungger, George H Booth, et al. 2022. The variational quantum eigensolver: a review of methods and best practices. *Physics Reports* 986 (2022), 1–128.

[55] Lisa Tse, Peter Mountney, Paul Klein, and Simone Severini. 2018. Graph cut segmentation methods revisited with a quantum algorithm. *arXiv preprint arXiv:1812.03050* (2018).

[56] Lisa Tse, Peter Mountney, Paul Klein, and Simone Severini. 2019. Graph Cut Segmentation Methods Revisited with a Quantum Algorithm. arXiv:1812.03050 [cs]

[57] Hayato Ushijima-Mwesigwa, Christian F. A. Negre, and Susan M. Mniszewski. 2017. Graph Partitioning Using Quantum Annealing on the D-Wave System. In *Proceedings of the Second International Workshop on Post Moores Era Supercomputing*. ACM, Denver CO USA, 22–29.

[58] Hayato Ushijima-Mwesigwa, Ruslan Shaydulin, Christian F. A. Negre, Susan M. Mniszewski, Yuri Alexeev, and Ilya Safro. 2021. Multilevel Combinatorial Optimization across Quantum Architectures. *ACM Transactions on Quantum Computing* 2, 1 (Feb. 2021), 1:1–1:29.

[59] Peter JM Van Laarhoven and Emile HL Aarts. 1987. Simulated annealing. In *Simulated annealing: Theory and applications*. Springer, 7–15.

[60] Nate Veldt, Austin R. Benson, and Jon Kleinberg. 2022. Hypergraph Cuts with General Splitting Functions. *SIAM Rev.* 64, 3 (Aug. 2022), 650–685.

[61] Joyce Jiyoung Whang, Rundong Du, Sangwon Jung, Geon Lee, Barry Drake, Qingqing Liu, Seonggoo Kang, and Haesun Park. 2020. MEGA: Multi-View Semi-Supervised Clustering of Hypergraphs. *Proc. VLDB Endow.* 13, 5 (Jan. 2020), 698–711.

[62] Yifan Wu, Ke Chen, Gang Chen, Dawei Jiang, Huan Li, and Lidan Shou. 2025. HyperMR: Efficient Hypergraph-enhanced Matrix Storage on Compute-in-Memory Architecture. *Proc. ACM Manag. Data* 3, 1 (Feb. 2025), 45:1–45:27.

[63] Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. 2007. Learning with Hypergraphs: Clustering, Classification, and Embedding. In *Advances in Neural Information Processing Systems*, Vol. 19. MIT Press.